

Assignment: Inference Engineering Week Task

Part A Theory

1. What's the difference between latency and throughput in model inference, which factor will influence model's performance more and how?

➔ 1) Latency : A particular model how much take to give a response to giving a one request. It is based on the request and response based cycle. The data who can tranfer from one node to the another node.

Ex. What is AI

For a single request model take 500ms to giving a response

Therefor the latency = 500ms

2) Throughput: A one model how may request or the token process in the particular amount of time. It is based on the time constraint to evaluate the amount of data can transfer in seconds.

Ex. A model name “Ravan” process 1000 req per second.

Therefore, Throughput is = 1000 req/sec

Q) The influence of model performance depends on the application we are using

1) Chatbot → More Latency → User wants an immediate response

2) Voice assistance → Latency → If the delay it feels annoying

3) Serving handles many request → Throughput → Need to handle many request

4) Interactive LLM application → Both → Fast response + many users.

2. Explain what KV cache is and how it helps with model's generation.

➔ KV Cache (Key – Value Cache) it is a temporary storage that we are using in inference engineering models to avoid repetitive task. If we use this then it can continuously calculate the key value pairs(vector) for tokens that have already been processed.

Ex.

- 1) Model processes "**What**" → stores its **K and V**.
- 2) Processes "**is**" → stores its K and V.
- 3) Processes "**AI**" → stores its K and V.
- 4) When generating the next token, it **reuses the stored K and V** instead of calculating them again.

Previous tokens → K/V stored in cache → Reuse during next-token generation

KV Cache stores the Key and Value representations of previously processed tokens during autoregressive generation, allowing the model to reuse them instead of recomputing them, which significantly improves inference speed at the cost of additional memory usage.

3. What is quantization, and what's the trade-off between int8/int4 quantization and model accuracy? Also give an example with calculations and formula of how quantization can be performed.

➔ It converting high precision model weights into lower precision format like it converting fp32 into lower floating points precisions

- It reduces the model size, it can improve the speed, if the model is too large for the small computer then it make easier to run on the small memory based computer....

Tradeoff : int8 = it using 8 bits(1byte) per weight, it is accurate than int4 , because we reduce the small precision.

int8 = Uses 4 bits per weight, gives much great memory savings but can cause more accuracy loss.

For a 7-billion parameter model:

FP16:

$$7\text{B} \times 16 \text{ bits} = 14 \text{ GB}$$

INT8:

$$7\text{B} \times 8 \text{ bits} = 7 \text{ GB}$$

INT4:

$$7\text{B} \times 4 \text{ bits} = 3.5 \text{ GB}$$

Quantization – Scale and Zero Point

Quantization converts a floating-point value into a smaller integer value such as INT8.

Formula:

$$X_q = \text{round}(X / S)$$

Where:

X_q = Quantized value

X = Actual/original value

S = Scale

Example:

Given:

$$X = 0.72$$

$$S = 0.0078$$

$$\text{Zero Point (Z)} = 0$$

Calculate Quantized Value:

$$X_q = \text{round}(X / S)$$

$$X_q = \text{round}(0.72 / 0.0078)$$

$$X_q = \text{round}(92.30)$$

$$X_q = 92$$

Therefore:

$$0.72 \text{ (Float32)} \rightarrow 92 \text{ (INT8)}$$

So, the original value 0.72 is stored as the quantized INT8 value 92.

4. Name one real inference serving framework (like vLLM or TensorRT-LLM) and briefly describe one optimization it does.

vLLM is an open-source inference serving framework designed for Large Language Models (LLMs). It helps deploy models and efficiently handle multiple user requests.

Optimization: PagedAttention

vLLM uses PagedAttention to efficiently manage the KV Cache during inference. Instead of storing the KV Cache in one large continuous memory space, it divides it into smaller blocks (pages).

These blocks can be allocated and reused efficiently for different requests. This reduces GPU memory wastage and allows vLLM to handle more requests at the same time.

Benefits:

- Better GPU memory utilization, supports more concurrent requests, reduces kv Cache memory wastage, improves inference throughput, helps serve multiple users efficiently

Example:

With pageAttention Multiple requests after that KV Cache and then Memory wastage and without pagedAttention Multiple requests then KV Cache divided into blocks then Efficient memory usage and then More requests